Multivariate Kalman Filter

Euan Riach

2026

1 Introduction

The Kalman Filter is an algorithm for filtering noise. This implementation is multivariate and linear. For non-linear systems an extended Kalman filter (EKF) is required. The EKF is not covered here.

2 Algorithm

This algorithm mostly comes from [2]. The Kalman filter repeats a two step process to estimate the state $\boldsymbol{x}$, an $n \times 1$ vector, of a system. First, prediction produces an a priori estimate using the filter's previous estimate. The filter then updates the a priori estimate by combining it with a measured value. The measurement does not need to cover all the state space.

2.1 Prediction

For notation, we will use $\hat{\boldsymbol{x}}_k^-$ to represent the a priori estimate of $\boldsymbol{x}$ at time k and $\hat{\boldsymbol{x}}_k$ for the a posteriori estimate. The filter requires a confidence in its current estimate, this is represented by the $n \times n$ error covariance matrix P_k, similarly P_k^- will represent the a priori estimate of P_k and P_k the a posteriori. The equations for the prediction step are

$$\hat{\boldsymbol{x}}_k^- = F\hat{\boldsymbol{x}}_{k-1} + B\boldsymbol{u}_k \tag{1}$$

$$P_k^- = FP_{k-1}F^T + Q. \tag{2}$$

The matrix $n \times n$ F is the state transition matrix, it is the linear equations used to model the system. $\boldsymbol{u}_k$ is a vector $l \times 1$ representing some driving force in time k and the matrix $n \times l$ B converts the measurement into the state space. Q is an $n \times n$ covariance matrix of the process noise.

2.2 Update

The filter then acquires an $m \times 1$ vector of some observation of the system which is subject to measurement variance. We first require the $n \times m$ Kalman gain

matrix, K. The Kalman Gain tells the filter how much weighting to give to the a priori estimate and observation. It is given by

$$K_k = P_k^- H^T (H P_k^- H^T + R)^{-1} \tag{3}$$

where H is the $m \times n$ matrix which transfers the observation to the state space and R is the $m \times m$ measurement noise covariance matrix. The filter can then use the Kalman gain to update the a priori estimates to give us a posteriori estimates:

$$\hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_k^- + K_k(z_k - H\hat{\boldsymbol{x}}_k^-) \tag{4}$$

$$P_k = (I - K_k H)P^-(I - KH)^T + KRK^T \tag{5}$$

Note: there are two formulas for P_k, we will use the Joseph form as it is more robust [1].
See appendix A for python implementation.

3 Demo - Falling Object

3.1 Setup

For this demo we will consider an object dropped from rest initially moving horizontally. It has downward acceleration due to gravity, it is assumed the object is close enough to the surface that g is constant. The state space will be

$$\boldsymbol{x} = \begin{pmatrix} x \\ y \\ v_x \\ v_y \\ a \end{pmatrix}$$

and the state transition matrix is

$$F = \begin{pmatrix} 1 & 0 & h & 0 & 0 \\ 0 & 1 & 0 & h & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & h \\ 0 & 0 & 0 & 0 & h \end{pmatrix}.$$

We will only observe position so

$$H = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

and our measurement variance matrix is the 2×2 identity scaled by our measurement variance. The process noise will be modelled as noise in the acceleration due to gravity, perhaps due to changes in air pressure. Thus the process variance matrix is 0 everywhere apart from q_{55}. The code for this demo can be found in the 'Demos' folder on GitHub titled 'Multivariate Kalman Filter Falling Object'.

3.2 Results

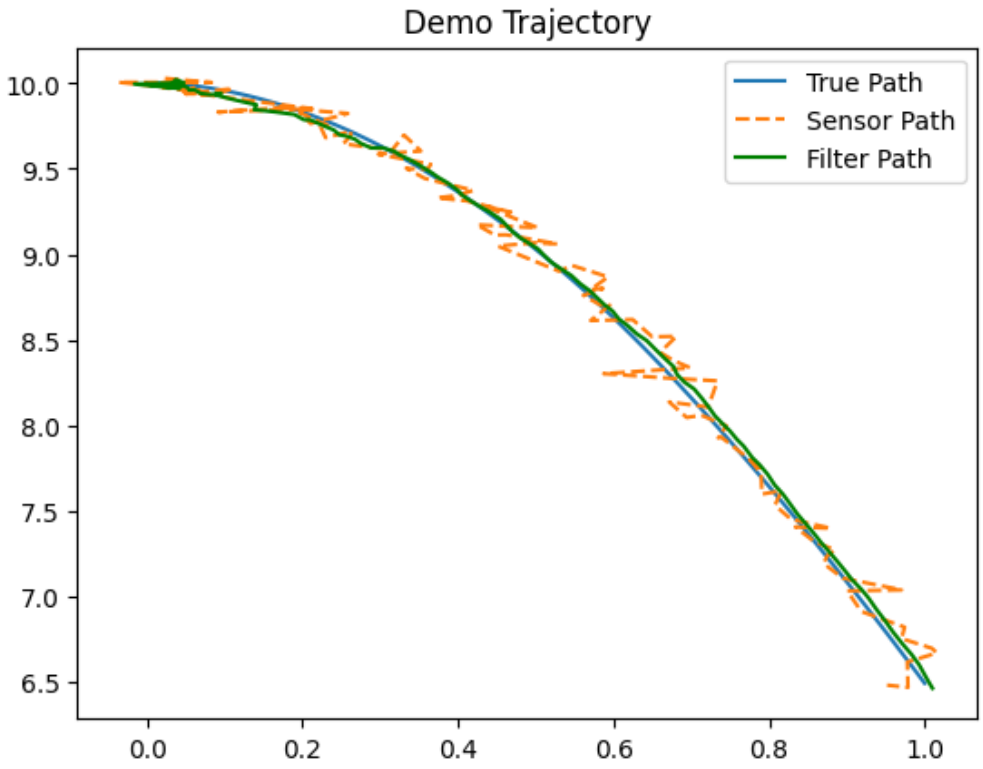


Figure 1: Kalman Gain for the State Space

The filter accurately follows the expected path only occasionally deviating. Calculating the root mean square error for both the filter and the sensor we got:
RMSE of filter: [0.02031406]
RMSE of sensor: [0.04465391]
This corresponds to a percentage decrease of 54.5% when using the filter.

4 Demo - Tracking a Bouncing Ball

4.1 Setup

To demonstrate the filter I will use the case of a ball moving in the xy plane confined to a box. Our state space will be

$$\boldsymbol{x} = \begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix}.$$

Let the motion be governed by the linear equations

$$x_k = x_{k-1} + v_x h$$
$$y_k = y_{k-1} + v_y h$$

where v_x and v_y represent the velocity in the x and y directions and are assumed to be constant and h is some time step. Hence, we can define the state transition matrix as

$$F = \begin{pmatrix} 1 & 0 & h & 0 \\ 0 & 1 & 0 & h \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

The ball is going to be confined in a box. If it hits the boundary it will bounce off the edge without losing energy.

When the ball is moving there will be added noise to the velocities to simulate small deviations due to external forces. Our observations will only be of the position so

$$z = \begin{pmatrix} x \\ y \end{pmatrix} + \text{noise}$$

and our measurement function will need to convert it to the state space, thus,

$$H = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$

When implementing the filter we must define the process covariance matrix appropriately. I have assumed there is no correlation between x and y measurements but there is correlation between position and velocity in the same axis. I have also added more variance in velocity to account for the ball bouncing at the boundary. Hence,

$$Q = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 2 \end{pmatrix} * q$$

where q is some scalar. The code used in this demo can be found in the demo folder on GitHub, titled 'Multivariate Kalman Filter Ball in Box'.

4.2 Results and Improvements

The filter works as intended. As can be seen from figure 1, the filter estimate accurately tracks the true position and ignores sensor noise. The limitations occur at the boundaries as there is a lag whenever the ball changes direction, this can be seen in the spikes in the residuals in figure 2. From figure 3, we see the Kalman Gain reaches a steady state after 250 iterations. The filter reaches a set confidence in its estimate. This is not ideal for this system. An improvement would be to have the filter add additional process noise whenever it estimates it is close to the boundaries, this would cause the Kalman gain to increase and give the filter more faith in the measurements when it is near a boundary.

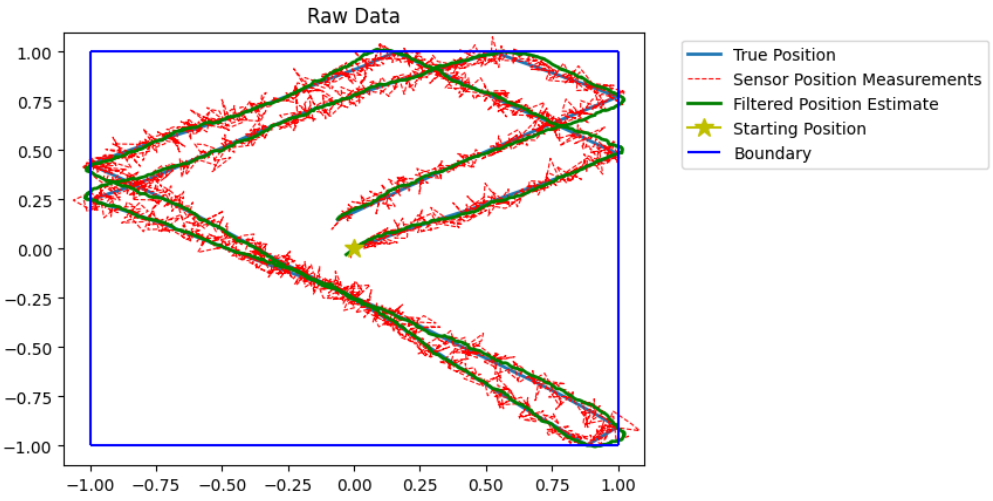


Figure 2: Path Taken

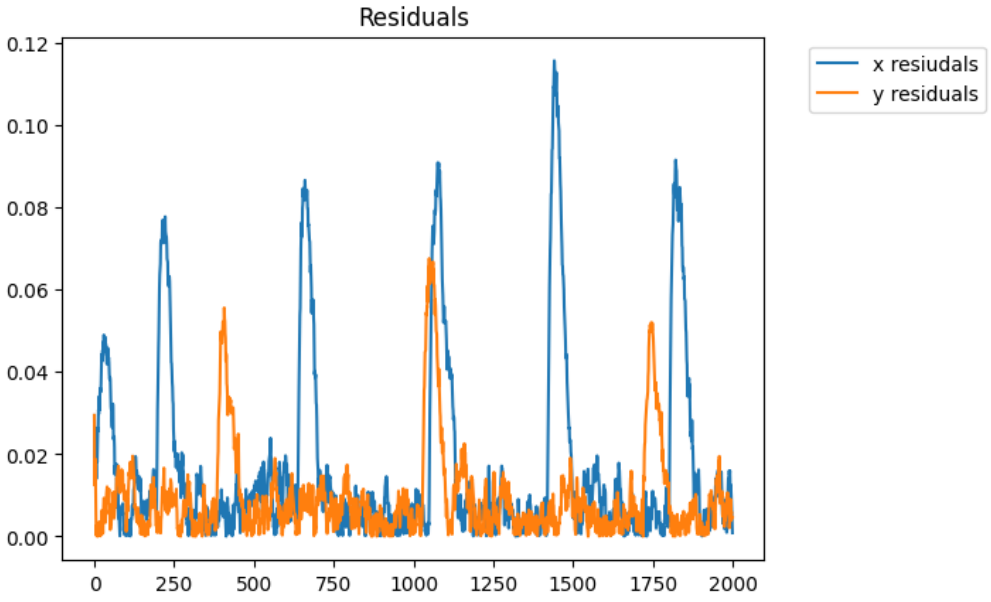


Figure 3: Residuals

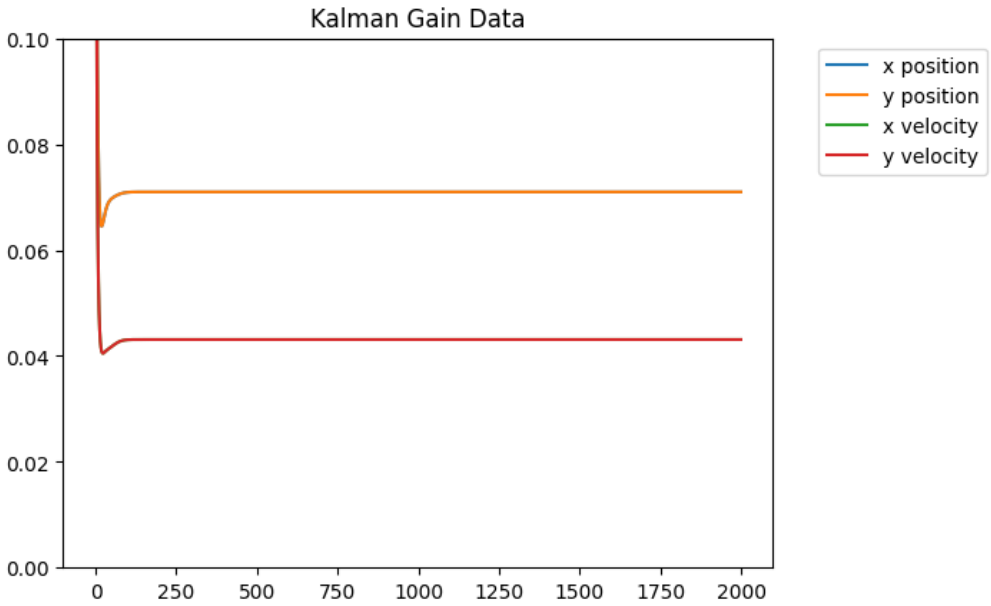


Figure 4: Kalman Gain for the State Space

5 Next Steps

Following on from this, building an extended Kalman filter would allow the modelling of more realistic systems. An EKF follows the same principle as the standard Kalman filter but requires linearising the function at each step using the Jacobian in place of the state transition matrix. Other applications of the multivariate Kalman filter include estimating volatility in equities by filtering out transaction noise and estimating the hedge ratio, β, of correlated assets.

References

[1] Roger Labbe. Kalman and bayesian filters in python, 2015.

[2] Greg Welch and Gary Bishop. An introduction to the kalman filter greg welch 1 and gary bishop 2 tr 95-041, 2001.

Appendix A - Multivariate Kalman Filter Class

```
import numpy as np

class KalmanFilter:
    def __init__(self, dim_x, dim_z, dim_u = 0):
        """
        The following are place holder objects and must be defined appropriately after
        calling the class.
        dim_x is the dimension of the state vector
        dim_z is the dimension of the measurement
        dim_u is the dimension of the driving force
        """
        self.x = np.zeros((dim_x, 1))       #State
        self.P = np.eye(dim_x)              #Error Covariance Matrix
        self.Q = np.eye(dim_x)              #Process Variance
        self.R = np.eye(dim_z)              #Measurement Variance
        self.u = np.zeros((dim_u, 1))       #Driving Force
        self.B = np.zeros((dim_x, dim_u))   #Control Transition Matrix
        self.F = np.zeros((dim_x, dim_x))   #State Transition Matrix
        self.H = np.zeros((dim_z, dim_x))   #Measurement Function
        self._I = np.eye(dim_x)             #Identity Matrix
        self.K = np.zeros((dim_x, dim_z))   #Kalman Gain
        self.dim_z = dim_z

    def predict(self):
        """
        Filter produces an a priori estimate of the next position off of the current
        estimate.
        """
        self.x = (self.F @ self.x) + (self.B @ self.u)
        self.P = (self.F @ self.P @ self.F.T) + self.Q

    def update(self, z):
        """
        Filter produces an a posteriori estimate by combining the a priori estimate
        with an observation.
        z - observation
        """
        z = np.asarray(z).reshape(self.dim_z, 1)
        self.K = self.P @ self.H.T @ np.linalg.inv(self.H @ self.P @ self.H.T + self.R)
        self.x = self.x + self.K @ (z - self.H @ self.x)
        self.P = (self._I - self.K @ self.H) @ self.P @ (self._I - self.K @ self.H).T
                + (self.K @ self.R @ self.K.T)
```